Statement of Purpose

My research interests are at the intersection of **natural language processing** and **programming languages**. By bridging the gap between natural and formal languages, we can develop systems that are able to convert high-level human specifications into programs, enabling non-expert users to achieve unprecedented levels of productivity. Despite recent advances in deep learning, this task remains challenging due to (1) the complexity of program syntax, (2) the semantic gap between natural language and programming languages, (3) the scarcity of high-quality data annotations, and (4) the lack of interpretability and trustworthiness in existing methods. In response to these challenges, I am focusing on two research topics: **How to accurately represent program structural information in neural models? How to integrate the strengths of symbolic and neural methods to synthesize programs with both scalability and interpretability?** During my study as an undergraduate at CUHK and a master's student at Tsinghua, I have already conducted several research as early steps in answering these questions. In the future, I would like to continue my research as a PhD student at UT Austin, and eventually accomplish my long-term goal of building systems that can connect humans with computing machines both reliably and efficiently.

Code Representation Learning: My passion for computer science was initially sparked by my programming study in my junior year courses. I was fascinated by the stark contrast between natural and formal language, and this eventually led me to my first research project on automatic commit message generation, which is essentially a task summarizing code changes into human language. The naturalness hypothesis, which suggests that software corpora have similar statistical properties to natural language corpora (Allamanis et al., 2018), inspired me to explore the use of NLP techniques for code representation learning. However, unlike previous works that treat code as plain sequences of tokens, I believe a program's structural features, such as branching and functional calls, are crucial for accurately representing the code semantics. This motivated our work published in **IEEE TSE**, in which I proposed extracting abstract syntax tree (AST) paths to explicitly encode the structural changes behind a code commit (Liu et al., 2020). Compared to the previous methods that neglect code structure, our method can capture the reasons behind code changes more accurately and generate more precise commit messages, achieving a 30.72% improvement in BLEU-4.

These exciting results encouraged me to delve deeper into code representation learning research. A question soon arose in my mind: how can we still capture the code structures when AST extraction is not feasible for incomplete code snippets or large-scale code refactoring? This led me to consider the potential of contextualized word embeddings like ELMo and BERT for representing code. These models have proven successful for natural language processing tasks by capturing the context in which words appear, and I wondered if a similar approach could be applied to code. After conducting a series of experiments on different self-supervised learning tasks, I proposed a contextualized code representation learning method that can exploit the structure of code from plain sequences (Nie et al., 2021). In this work published in **Neurocomputing**, I demonstrated that domain-specific contextualized representation learning, even without using any external corpus, can lead to significant improvements in downstream code-to-text tasks as well as better generalization under low-resource settings.

Program Synthesis & Semantic Parsing: Despite the progress I had made in mapping code to natural language, I found the opposite task of semantic parsing, i.e., converting natural language to code, yet remains challenging due to the complexity of program syntax. For example, many works aim to synthesize database queries from natural language using neural networks to facilitate non-expert users' interaction with structured data. However, I noticed that existing datasets are often inadequate in scale and fail to capture the multi-hop reasoning involved in complex query patterns. This led me to develop KoPL, a domain-specific language with a functional programming interface that explicitly expresses the reasoning process over structured knowledge. By formalizing a synchronous context-free grammar, I further synthesized ~120k [NL, KoPL, SPARQL] parallel corpora over the Wikidata knowledge base, which resulted in our work published at **ACL 2022**, the largest dataset to date for graph query language semantic parsing (Cao et el., 2022).

After realizing the current approaches' overreliance on data annotation, I began exploring how symbolic methods could aid in the synthesis of formal languages. Since neural methods struggle to synthesize syntactically correct programs,

and symbolic methods may not be able to handle diverse natural language inputs, why not divide the task and let neural and symbolic modules handle the sub-tasks they excel at? To connect a neural semantic parser with a compiler, I designed a novel intermediate representation (IR) that bridges the semantic gap between natural language and graph query languages (Nie et al., 2022). The IR is representable as a context-free grammar that is syntactically similar to natural language and preserves the structural semantics of formal languages. This allowed me to use a pretrained Seq2Seq model to precisely convert users' natural language specifications into the IR, which could then be losslessly compiled into various downstream graph query languages. From end to end, my proposed approach consistently showed stronger robustness in compositional generalization and low-resource settings. Furthermore, with the IR as a unified middleware, I also implement a source-to-source compiler that unlocks data interoperability by supporting the translation among different graph query languages. Eventually, this work has led to a paper just presented at **EMNLP 2022**.

Through this project, I discovered that semantic parsers and compilers have many similarities, as they both convert high-level language into low-level logical form. However, the rule-based analysis and conversion inside a compiler are generally transparent and reliable, whereas the deep learning models in neural semantic parsers are often treated as black boxes with little interpretability. To address this issue, in my latest **AAAI 2023** work done during my internship at Microsoft Research (Nie et al., 2022), I proposed to unveil the internal processing mechanism of pretrained language models (PLM). Specifically, I identified some atomic code structures that persist across different formal languages, and correspondingly designed intermediate supervision tasks to explicitly highlight the conversion of these "semantic anchors" alongside a PLM's fine-tuning. Consequently, the layer-wise hidden representations inside a PLM can be probed as human-readable outputs, which are extremely useful for interpreting the inner process of neural semantic parsing.

Future Plans: In recent years, while code pretrained models like CodeX have achieved impressive performance, these neural synthesis approaches still lack reliability. Therefore, having seen and developed models that combine neural networks and symbolic algorithms, I am interested in building more reliable, interpretable, and trustworthy program synthesis models by exploring **neurosymbolic programming systems**. For example, since real-world programming often involves trial and error, I plan to explore the **inductive synthesis** methods that incorporate **compile-time & runtime information** (e.g., dataflow, error message, execution result) and **human feedback** into the neural networks to generate and refine programs iteratively. Additionally, by training a neural network and then searching for a symbolic program whose behavior approximately matches the network's, I also hope to explore the **distillation of PLM knowledge into symbolic reasoning modules** with the help of domain-specific languages.

Why UT Austin: To accomplish these goals, I would be thrilled to pursue a PhD at the University of Texas at Austin, where many esteemed faculty and talented students are doing fascinating research that closely aligns with my interests. I am particularly interested in working with **Prof. Swarat Chaudhuri**, whose recent work on combining symbolic grammar and neural models for program synthesis perfectly matches my ambition to develop neurosymbolic programming systems that can achieve both scalability and reliability. I also hope to work with **Prof. Greg Durrett** and **Prof. Isil Dillig**, whose collaborative research on multimodal program synthesis with natural language specifications and I/O examples aligns well with my goals of connecting natural language with formal language. Following their research over time, I see UT Austin as a clear fit for my interests and the best place for me to pursue a PhD, and I am confident that the interdisciplinary and productive research environment at UT Austin can eventually equip me with the necessary skills to succeed in pursuing an academic career.

Miltiadis Allamanis, Earl T. Barr, Premkumar T. Devanbu, Charles Sutton. A survey of machine learning for big code and naturalness. In *ACM Computing Surveys*, 2018.

Shulin Cao, Jiaxin Shi, Liangming Pan, **Lunyiu Nie**, Yutong Xiang, Lei Hou, Juanzi Li, Hanwang Zhang, Bin He. KQA Pro: A Dataset with Explicit Compositional Programs for Complex Question Answering over Knowledge Base. In *Proc. of ACL*, 2022.

Shangqing Liu, Cuiyun Gao, Sen Chen, **Lunyiu Nie**, Yang Liu. ATOM: Commit Message Generation Based on Abstract Syntax Tree and Hybrid Reranking. In *IEEE Transactions on Software Engineering* (TSE), 2020.

Lunyiu Nie, Cuiyun Gao, Zhicong Zhong, Wai Lam, Yang Liu, Zenglin Xu. CoreGen: Contextualized Code Representation Learning for Commit Message Generation. In *Neurocomputing*, 2021.

Lunyiu Nie, Shulin Cao, Jiaxin Shi, Qi Tian, Lei Hou, Juanzi Li, Jidong Zhai. GraphQ IR: Unifying the Semantic Parsing of Graph Query Languages with One Intermediate Representation. In *Proc. of EMNLP*, 2022.

Lunyiu Nie, Jiuding Sun, Yanling Wang, Lun Du, Han Shi, Dongmei Zhang, Lei Hou, Juanzi Li, Jidong Zhai. Guiding the PLMs with Semantic Anchors as Intermediate Supervision: Towards Interpretable Semantic Parsing. In *Proc. of AAAI*, 2023.